

CMPS211 Advanced Programming Techniques

Collaborative Plain Text Editor

Overview

This project requires you to design and implement a simplified real-time collaborative plain text editor in Java. This application will allow multiple users to simultaneously edit the same document through shareable codes. The aim of this project is to:

- Develop a functional collaborative plain text editor with real-time editing and cursor tracking.
- Practice concepts you learned in the course

Specification

Document & Collaboration Management [20%]

- **Import/Export Files:** Users can import/export .txt files from/to their devices. The system must preserve line breaks and any word-level or character-level formatting (bold and italic). Imported files must be converted into the internal CRDT representation (Block CRDT + Character CRDT). Exported files must reconstruct the visible document state, including formatting.
- **Sharing:** Users can request shareable codes from the application server. Each file has 2 codes: one code is dedicated to editors and another is dedicated to read-only collaborators. Collaboration sessions start by sharing the codes with other users (over WhatsApp, email, phone, etc.). Viewer users cannot edit the document and cannot view the shareable codes.
- **File Management:** Allow users to create, open, rename, and delete files. All files must be stored persistently in a database. The document must be saved in its CRDT form (block structure and character structure), not as plain text.

Real-time Collaborative Editing [60%]

- **Block-Based + Character-Based Editing:**
 - The document must be structured as an ordered sequence of Blocks. A Block is defined as a uniquely identified logical unit containing a sequence of sentences spanning partially or fully up to 10 lines, with a minimum of 2 lines. Each block contains an ordered Character CRDT.
 - The system must support two types of operations:
 - **Block Operations:**
 - Insert block
 - Delete block
 - Split block
 - Modify block content
 - Copy and paste an entire block
 - Move block (reorder within the document)
 - Copy/paste content from one block to another
 - Block operations must be replicated consistently across all users and must remain deterministic under concurrent edits.
 - All Blocks operations must be applied using the Block-Level CRDT and must resolve conflicts deterministically.
- **Character Operations (inside a block):**
 - Character-by-character insertion

CMPS211 Advanced Programming Techniques*Collaborative Plain Text Editor*

- Character deletion
- Replace characters
- Paste text (treated as multiple insertions)
- Copy and paste partial content within or across blocks
- All character operations must be applied using the Character-Level CRDT and must resolve conflicts deterministically.
- **Formatting Support:**
 - Support bold and italic formatting at word-level and character-level granularity.
 - Formatting can be applied, removed, or toggled on any character range within a block.
 - Formatting operations must be replicated and conflict-resolved correctly under concurrency.
- **Concurrent Edits:**
 - Implement a tree-based Conflict-free Replicated Data Type (CRDT) algorithm to handle concurrency issues. The system must use a two-layer CRDT design:
 - Block-Level CRDT (for structural and ordering operations)
 - Character-Level CRDT (inside each block for text editing and formatting)
 - The implementation must correctly handle:
 - Concurrent character insertions at the same position
 - Concurrent character deletion and insertion at overlapping positions
 - Concurrent formatting on overlapping ranges
 - Concurrent block splits
 - Concurrent block merges
 - Block deletion while another user edits inside the block
 - Copy/paste operations occurring concurrently with edits
 - Conflict resolution must be deterministic and clearly documented in the report.
- **Real-time Updates:** Send/receive edits to/from other users in real-time. All edits must be relayed through a central server. Peer-to-peer communication is not allowed.
- **Cursor Tracking & User Presence:** Update the cursor positions of other users in the editor. Update the list of users who are currently present in the collaboration session. Cursor updates must occur in real-time.
- **Undo/Redo Changes:** Users can undo/redo latest actions at the character level or block level. The user can undo/redo changes made by other users. Your program should be able to undo/redo at least the latest 10 operations. Undo/redo must generate valid inverse CRDT operations and must function correctly under concurrent editing.

UI [20%]

Design a user-friendly user interface that includes the following:

- **Text Area:** Display a text area that shows the document and allows text editing.
- **Sharing Option:** Display the shareable codes to the user.
- **Bold and Italic:** Display Bold and Italic for the user to change text.
- **Collaboration:** Allow the user to join a collaboration session.
- **Import/Export Options:** Display a menu to allow the user to import/export files.
- **Files** could be saved in a database in server side and client side, each document should have a unique id.
- **Permission Handling:** Viewer users are not allowed to edit in the text area. Viewers cannot also view the shareable codes.
- **Cursor Display & User Presence:** Visually represent the cursors of other users in the editor. Cursors should be visually distinct (e.g., different colors). Display a list of active users. To restrict

CMPS211 Advanced Programming Techniques

Collaborative Plain Text Editor

the cursor colors, you may assume when designing the UI that the maximum number of concurrent editors per document is 4.

Bonus [6%]

- User comments: Editors can add comments to parts of the text.
The comment should be removed if the text is removed.
- Reconnection: Allow users a period of 5 minutes to reconnect if they experience a network drop. Remote edits missed should be sent to the user on reconnection. Local changes done should be sent after reconnection.
- Supporting version history and allow rolling back to previous document version

Implementation and Deliverables

Teams & Deadlines

All teams will deliver the project to the course classroom on **Monday (3th of May) at 2 pm**. Each team will discuss the project on the scheduled day and time.

Provide a zipped folder named **Team_<team number>.zip** containing the following materials:

- Your code files
- A readme.txt, explaining how to run your code
- A members.txt containing the names and IDs of each student in the group

Work in groups of 4 members.

Project Phases

The project will be delivered in **three phases** to help you progress steadily and avoid last-minute implementation issues.

Important:

These phase submissions are **ungraded deliverables**.

However, you **must submit working code for each phase**.

You may **submit partial but working code for each phase**.

Failure to submit code will result in grade **deductions from the final project mark**.

Phase 1: CRDT Deadline 28/3

Focus on:

- Character-Level CRDT
- Block-Level CRDT
- Insert/Delete operations
- Tombstones
- Deterministic ordering
- Basic local testing (no full UI required)

CMPS211 Advanced Programming Techniques

Collaborative Plain Text Editor

Phase2: UI+Networking Deadline 18/4

Focus on:

- Text editor UI
- Real-time communication (WebSocket or equivalent)
- Sending/receiving CRDT operations
- Cursor tracking
- Basic collaboration session

Phase 3: Final delivery backend+Full integration Deadline Monday (3th of May) at 2 pm

Focus on:

- Database persistence
- File management
- Sharing codes
- Permission handling (editor/viewer)
- Full system integration
- Stable, working application

Implementation

- Java should be used for all logic.
- **Concurrency:** Multiple users should be able to edit the same document concurrently, and multiple documents can be active at once. The system must handle these concurrent edits in a non-blocking manner to maintain responsiveness.
- Use any framework for UI as long as the logic and data handling is in Java. **Fancy UI is not required and it is not a bonus.**
- It is your responsibility to select the best data structures, algorithms, techniques and tools that enhance the project performance
- You should handle network problems and exceptions in your program (connection errors, wrongly formatted messages..etc.)

Libraries and Packages Regulations

You could use a library only if you followed **all** of the points below:

- You can use libraries for UI, networking, JSON handling, html handling, ID generation, and data persistence.
- The core CRDT implementation (data structures and the logic) **must** be your own code.
- You are responsible for the library accuracy. (If it does a bad job, then it is your responsibility).
- You should understand how the library works
- If you are not sure if library usage is allowed or not, contact the instructors.

Evaluation and Grading Criteria

- The project is graded as a whole and the discussion decides what is the grade of each student (there's no piggybacking)
- Any delay in delivery will be penalized by losing 10% of the grade for each late day
- Code must be original, and must not be copied or shared from any other source, or generated by any AI, except as provided by the class instructors.

CMPS211 Advanced Programming Techniques

Collaborative Plain Text Editor

- Note: A plagiarized or AI-generated project means ZERO in the project and deduct some grades from the other coursework.

~~~~~ Good Luck ~~~~~